Algorithmique I

Chapitre 2 · Algorithmes linéaires

Corrigé des exercices 1 à 10 (2026-2027)

Les solutions emploient le LDA du syllabus actuel : `void`, `int`, `float`, `Boolean`, `String`, `read`, `print`, affectation `=`, `DIV`, `MOD`. Les données sont supposées respecter les préconditions précisées. Les exercices ne nécessitent ni `if` ni boucle.

Exercice 1 : Choix de type

Énoncé : choisir le type approprié pour six informations. *[1, p. 21]*.

Donnée	Type	Justification
Âge (années révolues)	`int`	Valeur entière
Poids	`float`	Mesure éventuellement décimale
Nom	`String`	Texte
Code postal belge	`int`	Convention simplifiée du cours
Numéro de maison	`String`	Peut contenir un suffixe, tel que « 12bis »
Adresse électronique	`String`	Texte avec caractères spéciaux

Un `Boolean` représenterait une information exclusivement vraie ou fausse, pas l'une des six données. Un type ne garantit pas la validité de la donnée.

Exercice 2 : On trace !

Données lues : `4`, `10`. *[1, p. 21]*.

Ligne	a	b	c
2 - Déclarations	-	-	-
3 - `read a, b`	4	10	-
4 - `c = a + 2*b`	4	10	24
5 - `a = c DIV b`	2	10	24
6 - `b = b MOD a`	2	0	24

Affichage (ligne 7) : `2 0 24`. Le calcul de la ligne 6 utilise la valeur actualisée `a=2`.

Exercice 3 : On retrace !

Données lues : `2`, puis `3`. *[1, p. 21]*.

Ligne	x	y
2 - Déclarations	-	-
3 - `read x`	2	-
4 - `x = 2*x`	4	-
5 - `y = 3*x`	4	12
6 - `read y`	4	3
7 - `y = 3*y`	4	9

Affichage (ligne 8) : `1`, car `9 MOD 4 = 1`. La seconde lecture de `y` écrase la valeur `12`. Une variable déclarée mais non initialisée ne vaut pas automatiquement zéro.

Exercice 4 : Échange de variables

Objectif : échanger les valeurs réellement mémorisées dans `x` et `y`. *[1, p. 22]*.

```
void echange()
    int x, y, temporaire
    read x, y
    temporaire = x
    x = y
    y = temporaire
    print x, y
```

Pour `x = 30`, `y = 40` : après sauvegarde `(30,40,30)` ; après `x = y` : `(40,40,30)` ; après restauration : `(40,30,30)` dans l'ordre `(x,y,temporaire)`.

Affichage : `40 30`. Sans la variable temporaire, la première valeur serait perdue.

Exercice 5 : Surface du triangle

Entrées : base et hauteur en centimètres. **Sortie :** aire en cm². *[1, p. 22]*.

```
void surfaceTriangle()
    float base, hauteur, aire
    read base, hauteur
    aire = base*hauteur/2
    print aire, " cm²"
```

Calcul : `6*4/2 = 12`. **Affichage exigé par l'exemple :** `12 cm²` (sans phrase supplémentaire). Les longueurs doivent être non négatives et exprimées dans la même unité ; pour un triangle non dégénéré elles sont positives. La division `/` conserve les résultats fractionnaires.

Exercice 6 : En voiture !

Entrées : distance en km, temps en minutes. **Sortie :** vitesse moyenne en km/h. *[1, p. 22]*.

```
void vitesseMoyenne()
    float distanceKm, minutes, vitesseKmH
    read distanceKm, minutes
    vitesseKmH = 60*distanceKm/minutes
    print vitesseKmH
```

Formule : `v = d/(t/60) = 60*d/t`. **Contrôle :** 150 km en 90 min donnent 100 km/h. Précondition : `distanceKm >= 0` et `minutes > 0`. Ne pas calculer une vitesse avec un temps nul.

Exercice 7 : Histoire de chiffres

Entrée : entier compris entre 100 et 999. *[1, p. 22]*.

```
void extraireChiffres()
    int nombre, centaines, dizaines, unites
    read nombre
    centaines = nombre DIV 100
    dizaines = (nombre DIV 10) MOD 10
    unites = nombre MOD 10
    print "le chiffre des centaines est ", centaines
    print "le chiffre des dizaines est ", dizaines
    print "le chiffre des unités est ", unites
```

Pour `723` : centaines `7`, dizaines `2`, unités `3`. **Contrôle :** `100*7 + 10*2 + 3 = 723`. `DIV` enlève des chiffres de droite ; `MOD 10` extrait le dernier chiffre.

Exercice 8 : Le nombre miroir

Entrée : entier de 100 à 999. *[1, p. 23]*.

```
void nombreMiroir()
    int nombre, c, d, u, miroir
    read nombre
    c = nombre DIV 100
    d = (nombre DIV 10) MOD 10
    u = nombre MOD 10
    miroir = 100*u + 10*d + c
    print miroir
```

Essais : `189` → `981` ; `752` → `257` ; `120` → `21` ; `100` → `1`. Les zéros placés en tête du résultat numérique ne sont pas affichés : `021` est le nombre entier 21.

Exercice 9 : Conversion HMS → S

Entrées : heures, minutes et secondes d'une durée. *[1, p. 23]*.

```
void dureeEnSecondes()
    int heures, minutes, secondes, total
    read heures, minutes, secondes
    total = 3600*heures + 60*minutes + secondes
    print total, " secondes"
```

Contrôle : `12*3600 + 3*60 + 40 = 43420` secondes. Domaine : heures ≥ 0 ; minutes et secondes entre 0 et 59.

Exercice 10 : Conversion S → HMS

Entrée : nombre entier de secondes d'une durée, ≥ 0. *[1, p. 23]*.

```
void secondesEnDuree()
    int total, heures, minutes, secondes
    read total
    heures = total DIV 3600
    minutes = (total MOD 3600) DIV 60
    secondes = total MOD 60
    print heures, "h ", minutes, "' ", secondes, "''"
```

Pour `43420` : `12h 3' 40''`. **Contrôle inverse :** `3600*12 + 60*3 + 40 = 43420`. Une durée peut dépasser 24 heures : ne pas réduire automatiquement ses heures modulo 24.

Références

[1] ESI, HE2B, *Algorithmique I : syllabus 2026-2027*, Bruxelles, Belgique, 2026.